

UNIT 5

LEETCODE EXERCISE

1. Product Sales Analysis I

Problem Summary

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

(sale_id, year) is the primary key of this table.
product_id is a foreign key to Product table.
Note that the price is per unit.

Table: Sales

Column Name	Type
product_id	int
product_name	varchar

product_id is the primary key of this table.

Task

Write an SQL query that reports the product_name, year, and price for each sale_id in the Sales table.

Solution

[Code](#) | [Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select p.product_name, s.year, s.price
3 from Sales s
4 left join Product p
5 on s.product_id = p.product_id
```

✓ Testcase | > Test Result

Accepted Runtime: 243 ms

✓ Case 1

Input

Sales =

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product =

product_id	product_name
100	Nokia
200	Apple
300	Samsung

Output

product_name	year	price
Nokia	2008	5000
Nokia	2009	5000
Apple	2011	9000

Explanation:

- The Sales table contains sales details.
- The Product table contains product names.
- Use INNER JOIN to connect both tables using product_id.
- Use SELECT to display the product name, year, and price.

2. Sales Analysis III

Problem Summary

Table: Product

Column Name	Type
product_id	int
product_name	varchar
unit_price	int

product_id is the primary key of this table.

Table: Sales

Column Name	Type
seller_id	int
product_id	int
buyer_id	int
sale_date	date
quantity	int
price	int

This table has no primary key, it can have repeated rows.

product_id is a foreign key to Product table.

Task

Write an SQL query that reports the products that were only sold in spring 2019. That is, between 2019-01-01 and 2019-03-31 inclusive.

Solution

[Code](#) | [Description](#) | [Editorial](#) | [Solutions](#) | [Submissions](#)

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select p.product_id, p.product_name
3 from Product p
4 left join Sales s
5 on p.product_id = s.product_id
6 group by p.product_id
7 having sum(s.sale_date between '2019-01-01' and '2019-03-31') = count(s.sale_date)
```

✓ Testcase | >_ Test Result

Accepted Runtime: 106 ms

✓ Case 1

Input

Product =

product_id	product_name	unit_price
1	S8	1000
2	G4	800
3	iPhone	1400

Sales =

seller_id	product_id	buyer_id	sale_date	quantity	price
1	1	1	2019-01-21	2	2000
1	2	2	2019-02-17	1	800
2	2	3	2019-06-02	1	800
3	3	4	2019-05-13	2	2800

Output

product_id	product_name
1	S8

Explanation

- This query selects products that were sold during the first three months of 2019.
- It removes products that were also sold before January or after March 2019 and displays the product ID and product name.

3. Product Price At A Given Date

Problem Summary

Table: Products

Column Name	Type
product_id	int
new_price	int
change_date	date

(product_id, change_date) is the primary key (combination of columns with unique values) of this table.

Each row of this table indicates that the price of some product was changed to a new price at some date.

Initially, all products have price 10.

Task

Write a solution to find the prices of all products on the date 2019-08-16.

Solution

[Code](#) | [Editorial](#) | [Solutions](#) | [Submissions](#) | [Description](#)

MySQL Auto

```
1 # Write your MySQL query statement below
2 WITH LatestPrice AS (
3     SELECT
4         product_id,
5         new_price,
6         ROW_NUMBER() OVER (
7             PARTITION BY product_id
8             ORDER BY change_date DESC
9         ) AS rn
10    FROM Products
11   WHERE change_date <= '2019-08-16'
12 )
13
14 SELECT
15     p.product_id,
16     COALESCE(lp.new_price, 10) AS price
17 FROM
18     (SELECT DISTINCT product_id FROM Products) p
19 LEFT JOIN LatestPrice lp
20     ON p.product_id = lp.product_id
21    AND lp.rn = 1;
```

Testcase | [Test Result](#)

Accepted Runtime: 69 ms

Case 1

Input

Products =

product_id	new_price	change_date
1	20	2019-08-14
2	50	2019-08-14
1	30	2019-08-15
1	35	2019-08-16
2	65	2019-08-17
3	20	2019-08-18

Output

```
| product_id | price |
|-----|-----|
| 1          | 35    |
| 2          | 50    |
| 3          | 10    |
```

Explanation

- The query finds the price of every product on August 16, 2019.
- It selects the latest price change on or before that date.
- If a product's price was not changed by that date, its initial price of 10 is displayed.

4. Immediate Food Delivery II

Problem Summary

Table: Delivery

Column Name	Type
delivery_id	int
customer_id	int
order_date	date
customer_pref_delivery_date	date

delivery_id is the column of unique values of this table.

The first order of a customer is the order with the earliest order date that the customer made. It is guaranteed that a customer has precisely one first order.

Task

Write a solution to find the percentage of immediate orders in the first orders of all customers, rounded to 2 decimal places.

Solution

[Code](#) | [Editorial](#) | [Solutions](#) | [Submissions](#) | [Description](#)

MySQL   Auto

```
1  # Write your MySQL query statement below
2  WITH FirstOrders AS (
3      SELECT *,
4          ROW_NUMBER() OVER (
5              PARTITION BY customer_id
6              ORDER BY order_date
7          ) AS rn
8      FROM Delivery
9  )
10
11 SELECT
12     ROUND(
13         100.0 *
14         SUM(CASE
15             WHEN order_date = customer_pref_delivery_date
16             THEN 1
17             ELSE 0
18         END)
19         / COUNT(*),
20         2
21     ) AS immediate_percentage
22 FROM FirstOrders
23 WHERE rn = 1;
```

Accepted Runtime: 96 ms

Case 1

Input

Delivery =

delivery_id	customer_id	order_date	customer_pref_delivery_date
1	1	2019-08-01	2019-08-02
2	2	2019-08-02	2019-08-02
3	1	2019-08-11	2019-08-12
4	3	2019-08-24	2019-08-24
5	3	2019-08-21	2019-08-22
6	2	2019-08-11	2019-08-13
7	4	2019-08-09	2019-08-09

Output

immediate_percentage
50

Explanation:

- The customer id 1 has a first order with delivery id 1 and it is scheduled.
- The customer id 2 has a first order with delivery id 2 and it is immediate.
- The customer id 3 has a first order with delivery id 5 and it is scheduled.
- The customer id 4 has a first order with delivery id 7 and it is immediate.
- Hence, half the customers have immediate first orders.

5.Monthly Transactions I

Problem Summay

Table: Transactions

Column Name	Type
id	int
country	varchar
state	enum
amount	int
trans_date	date

id is the primary key of this table.

The table has information about incoming transactions.

The state column is an enum of type ["approved", "declined"].

Task

Write an SQL query to find for each month and country, the number of transactions and their total amount, the number of approved transactions and their total amount.

Solution

[Code](#) | [Editorial](#) | [Solutions](#) | [Submissions](#) | [Description](#)

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     DATE_FORMAT(trans_date, '%Y-%m') AS month,
4     country,
5     COUNT(*) AS trans_count,
6     SUM(CASE
7         WHEN state = 'approved' THEN 1
8         ELSE 0
9     END) AS approved_count,
10    SUM(amount) AS trans_total_amount,
11    SUM(CASE
12        WHEN state = 'approved' THEN amount
13        ELSE 0
14    END) AS approved_total_amount
15 FROM Transactions
16 GROUP BY DATE_FORMAT(trans_date, '%Y-%m'), country;
```

Testcase | > Test Result

Accepted Runtime: 82 ms

Case 1

Input

Transactions =

id	country	state	amount	trans_date
121	US	approved	1000	2018-12-18
122	US	declined	2000	2018-12-19
123	US	approved	2000	2019-01-01
124	DE	approved	2000	2019-01-07

Output

month	country	trans_count	approved_count	trans_total_amount	approved_total_amount
2018-12	US	2	1	3000	1000
2019-01	US	1	1	2000	2000
2019-01	DE	1	1	2000	2000

Explanation

- The query groups transactions by month and country.
- It calculates the total number of transactions, the number of approved transactions, the total transaction amount, and the total amount of approved transactions using COUNT(), SUM(), CASE, and GROUP BY.

6. Last Person to Fit in the Bus

Problem Summary

Table: Queue

Column Name	Type
person_id	int
person_name	varchar
weight	int
turn	int

person_id column contains unique values.

This table has the information about all people waiting for a bus.

The person_id and turn columns will contain all numbers from 1 to n, where n is the number of rows in the table.

turn determines the order of which the people will board the bus, where turn=1 denotes the first person to board and turn=n denotes the last person to board.
weight is the weight of the person in kilograms.

Task

Write a solution to find the person_name of the last person that can fit on the bus without exceeding the weight limit. The test cases are generated such that the first person does not exceed the weight limit.

Solution

[Code](#) | [Editorial](#) | [Solutions](#) | [Submissions](#) | [Description](#)

MySQL   Auto

```
1  # Write your MySQL query statement below
2  WITH RunningTotal AS (
3      SELECT
4          person_name,
5          turn,
6          SUM(weight) OVER (ORDER BY turn) AS total_weight
7      FROM Queue
8  )
9
10 SELECT person_name
11 FROM RunningTotal
12 WHERE total_weight <= 1000
13 ORDER BY turn DESC
14 LIMIT 1;
```

✓ Testcase | >_ Test Result

Accepted Runtime: 94 ms

✓ Case 1

Input

Queue =

person_id	person_name	weight	turn
5	Alice	250	1
4	Bob	175	5
3	Alex	350	2
6	John Cena	400	3
1	Winston	500	6
2	Marie	200	4

Output

person_name
John Cena

Explanation

The following table is ordered by the turn for simplicity.

7. Restaurant Growth

Problem Summary

Table: Customer

Column Name	Type
customer_id	int
name	varchar
visited_on	date
amount	int

In SQL, (customer_id, visited_on) is the primary key for this table.

This table contains data about customer transactions in a restaurant. `visited_on` is the date on which the customer with ID (`customer_id`) has visited the restaurant. `amount` is the total paid by a customer.

Task

Compute the moving average of how much the customer paid in a seven days window (i.e., current day + 6 days before). `average_amount` should be rounded to two decimal places.

Solution

[Code](#) | [Editorial](#) | [Solutions](#) | [Submissions](#) | [Description](#)

MySQL   Auto

```
1  # Write your MySQL query statement below
2  WITH daily AS (
3      SELECT
4          visited_on,
5          SUM(amount) AS daily_total
6      FROM Customer
7      GROUP BY visited_on
8  )
9
10 SELECT
11     d1.visited_on,
12     SUM(d2.daily_total) AS amount,
13     ROUND(SUM(d2.daily_total) / 7, 2) AS average_amount
14 FROM daily d1
15 JOIN daily d2
16     ON d2.visited_on BETWEEN DATE_SUB(d1.visited_on, INTERVAL 6 DAY)
17     AND d1.visited_on
18 GROUP BY d1.visited_on
19 HAVING COUNT(d2.visited_on) = 7
20 ORDER BY d1.visited_on;
```

✓ Testcase | > Test Result

Accepted Runtime: 93 ms

✓ Case 1

Input

Customer =

customer_id	name	visited_on	amount
1	Jhon	2019-01-01	100
2	Daniel	2019-01-02	110
3	Jade	2019-01-03	120
4	Khaled	2019-01-04	130
5	Winston	2019-01-05	110
6	Elvis	2019-01-06	140
7	Anna	2019-01-07	150
8	Maria	2019-01-08	80
9	Jaze	2019-01-09	110
1	Jhon	2019-01-10	130
3	Jade	2019-01-10	150

Output

visited_on	amount	average_amount
2019-01-07	860	122.86
2019-01-08	840	120
2019-01-09	840	120
2019-01-10	1000	142.86

Explanation

- 1st moving average from 2019-01-01 to 2019-01-07 has an average_amount of $(100 + 110 + 120 + 130 + 110 + 140 + 150)/7 = 122.86$
- 2nd moving average from 2019-01-02 to 2019-01-08 has an average_amount of $(110 + 120 + 130 + 110 + 140 + 150 + 80)/7 = 120$

- 3rd moving average from 2019-01-03 to 2019-01-09 has an average_amount of $(120 + 130 + 110 + 140 + 150 + 80 + 110)/7 = 120$
- 4th moving average from 2019-01-04 to 2019-01-10 has an average_amount of $(130 + 110 + 140 + 150 + 80 + 110 + 130 + 150)/7 = 142.86$